1. Execution Summary

The objective of this task was to implement robust data management and error-handling protocols using the Java Collections Framework. The "Central Library Management System" demonstrates how to maintain synchronized data across different collection types while ensuring system stability through custom exception handling.

By integrating ArrayList, HashMap, and HashSet, the application provides a high-performance environment for data storage and retrieval, simulating a real-world enterprise backend.

2. System Module: Library Resource Management

This module utilizes specific Java Collections and Exception Handling techniques to manage library assets:

- **Data Structures:** * **ArrayList:** Used to maintain an ordered catalog of book titles for sequential display.
    - **HashMap:** Implemented for high-speed resource retrieval by mapping unique 3-digit IDs to book records.
    - **HashSet:** Employed to store unique library categories, automatically preventing duplicate department entries.
- **Exception Handling Architecture:**
    - **Custom Exception (InvalidBookException):** A specialized class created to handle library-specific errors, such as missing resource IDs or duplicate registration attempts.
    - **Try-Catch Blocks:** Used to wrap operational logic, preventing system crashes during invalid user input or data conflicts.

    - **Finally Block:** Ensures that the system session is securely refreshed and the Scanner resource is closed, regardless of whether an error occurred.
- **Synchronized Updates:** The module demonstrates complex logic where new data is validated via a custom exception before being simultaneously updated across both the ArrayList and HashMap.

Source Code: (LibrarySystem.java)

```
import java.util.*;

// Custom Exception (Basic Level) class
InvalidBookException extends Exception {
public InvalidBookException(String message) {
super(message);
   }
}

public class LibrarySystem {     public
static void main(String[] args) {
      // Collections initialization with simple names
      List<String> bookList = new ArrayList<>();
      Map<Integer, String> bookMap = new HashMap<>();
```

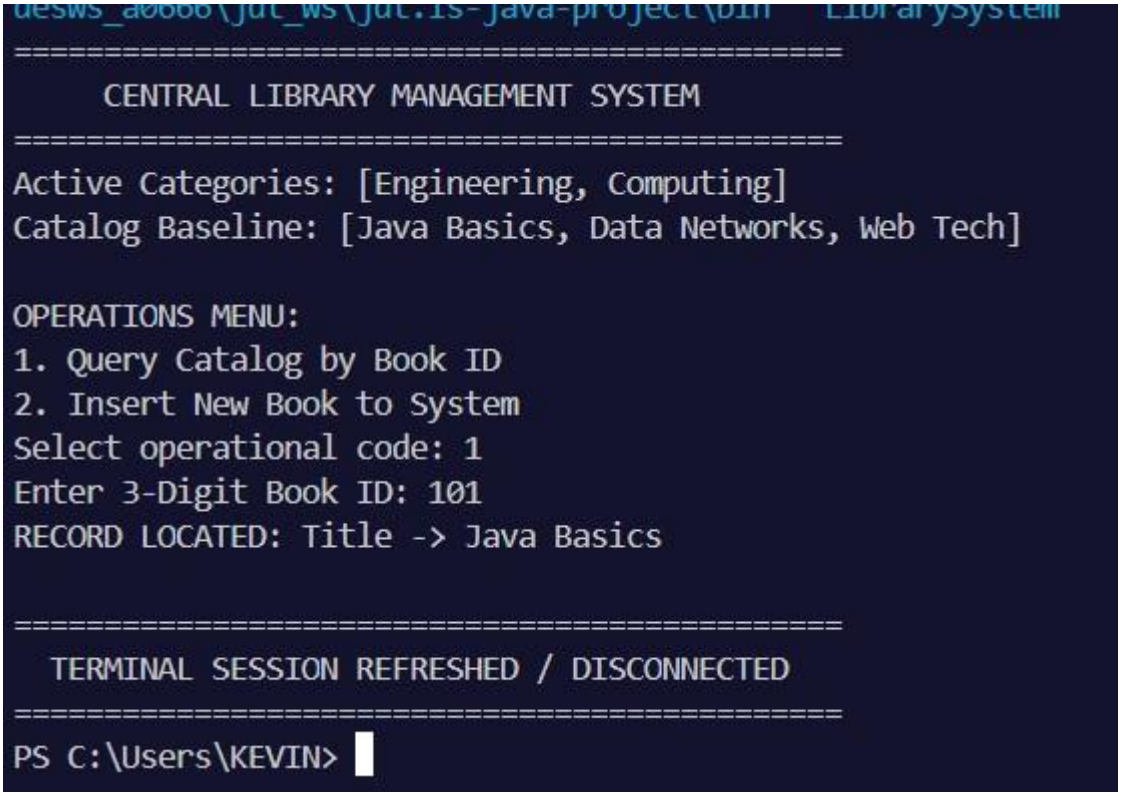
```
        Set<String> sections = new HashSet<>();

        Scanner scan = new Scanner(System.in);
       // Pre-populating some database records
bookList.add("Java Basics");       bookList.add("Data
Networks");
        bookList.add("Web Tech");

        bookMap.put(101, "Java Basics");
bookMap.put(102, "Data Networks");
bookMap.put(103, "Web Tech");

        // Unique library categories
sections.add("Computing");
sections.add("Engineering");
        sections.add("Computing"); // Duplicate entry (will be
autofiltered)

        System.out.println("================================
==========");
        System.out.println("     CENTRAL LIBRARY MANAGEMENT
SYSTEM       ");
        System.out.println("================================
==========");
```

```
        System.out.println("Active Categories: " + sections);
        System.out.println("Catalog Baseline: " + bookList);
       // Exception handling block for active operations
try {
          System.out.println("\nOPERATIONS MENU:");
           System.out.println("1. Query Catalog by Book ID");
           System.out.println("2. Insert New Book to System");
System.out.print("Select operational code: ");           int
action = scan.nextInt();           scan.nextLine(); // Clear
input buffer

           if (action == 1) {
              System.out.print("Enter 3-Digit Book ID: ");
int id = scan.nextInt();

              // Validating ID existence via Exception              if
(!bookMap.containsKey(id)) {                throw new
InvalidBookException("CRITICAL ERROR: Resource ID " + id + "
not registered in catalog.");
              } else {
                 System.out.println("RECORD LOCATED: Title -> " +
bookMap.get(id));
              }
           }
```

```
        else if (action == 2) {
            System.out.print("Enter New Book ID: ");
int newId = scan.nextInt();             scan.nextLine();
// Clear buffer

            // Check if ID is already taken              if
(bookMap.containsKey(newId)) {                  throw new
InvalidBookException("ABORT EXECUTION: Resource ID " + newId
+ " is already assigned.");
            }

            System.out.print("Enter Book Title: ");
            String title = scan.nextLine();

            // Dynamic updates across multiple collections
bookMap.put(newId, title);              bookList.add(title);

            System.out.println("SUCCESS: Catalog arrays updated
synchronized.");
            System.out.println("Updated Catalog: " + bookList);
        }
else {
            System.out.println("WARNING: Operation code
unrecognized.");
```

```
            }
        }
        catch (InvalidBookException e) {
            System.out.println(e.getMessage());
        }
        catch (InputMismatchException e) {
            System.out.println("SYSTEM ERROR: Invalid alphanumeric format parsed. Session locked.");
        }
finally {
            System.out.println("\n================================================");
            System.out.println("  TERMINAL SESSION REFRESHED / DISCONNECTED   ");
            System.out.println("================================================");
            scan.close();
        }
    }
}
```

Output:



3. Conclusion

Task III successfully demonstrates the transition from basic logic to advanced data management. By utilizing the Collections Framework, the system achieves efficient $O(1)$ search time complexity via hashing while maintaining data order through list structures. The implementation of custom exceptions ensures that the application is "fault-tolerant," a critical requirement for professional Java development.